GenAI Character Design Studio

Case Study 65 - GenAI for Video Game Character Design

An AI-assisted game-design workflow with consistency checking, guardrails and human creative control

Name: Suraj T
USN: 1DS23AI058

1. Abstract

Game studios want to use generative AI to speed up character design, but raw model output is unreliable: it breaks the game's rules and lore, can imitate copyrighted characters, can produce content unsuitable for the target age rating, costs money per call, and risks taking creative control away from designers. This project builds a working AI-assisted design workflow that generates character concepts, dialogue and story variations, and wraps the generator in guardrails: a machine-readable Game Bible, input and output safety filters, a copyright detector, a rule-based consistency checker with automatic repair, cost controls, and a mandatory human review step with full version history. The system runs fully offline with no API key, and can switch to a free local LLM (Ollama) without code changes.

2. Problem statement

A game-development studio wants to use GenAI to generate character concepts, dialogue and story variations. The problem is to develop an AI-assisted game-design workflow that generates content while maintaining consistency with game rules and narrative constraints, while also considering copyright, inappropriate content, computational cost and human creative control.

Objectives

- Generate complete, usable character packages (concept, abilities, backstory, art prompt, dialogue, arcs).
- Ground all generation in an editable Game Bible and measure consistency automatically.
- Detect and fix rule violations without regenerating everything.
- Block unsafe or manipulative input and output; flag likely copyright problems.
- Make cost visible and controllable.
- Keep the human designer as the final decision-maker, with an audit trail.

3. System architecture

Game Bible feeds the Generator and the Consistency checker. If the checker finds errors, Auto-repair regenerates only the broken fields and loops back through the guardrails (max 2-3 rounds).

Module	File	Responsibility
Game Bible	studio/game_bible.py	World rules, lore, factions, tech level, banned terms (JSON, editable in UI)
Generator	studio/engines.py	Offline procedural engine or local LLM (Ollama); same interface
Prompts	studio/prompts.py	Bible-grounded prompts for character, repair, dialogue, arcs
Guardrails	studio/safety.py	Content filter, prompt-injection filter, copyright detector
Consistency	studio/consistency.py	Rules R1-R7, score out of 100, error/warning severity
Pipeline	studio/pipeline.py	Orchestrates all stages and the auto-repair loop
Cost	studio/cost.py	Token metering, caching, model routing, budget cap
Review	studio/review.py	Versioned human review: approve, edit, reject, diff

Export	studio/export.py	JSON for game engines, PDF character sheet with provenance
UI	app.py	Streamlit web app with 7 tabs

4. How each requirement is met

Requirement	Design decision
Character concepts	Name, appearance, palette, abilities, signature item, backstory, motivation, flaw, catchphrase, a concept-art portrait drawn from the validated data (so the art matches the Game Bible) and a ready-to-use prompt for an image model.
Dialogue & story variations	Situation-aware dialogue in the character's personality voice; five story-arc types (Redemption, Betrayal, Tragic Sacrifice, Rise to Power, Hidden Heir).
Consistency with rules & narrative	A machine-readable Game Bible grounds every prompt; seven automated rules score every output; violated fields are regenerated automatically.
Copyright	Fuzzy name matching against protected characters, franchise-term detection, signature-trait combinations and 'in the style of <artist/studio>' detection.
Inappropriate content	Age-rating content filter on both designer input and model output, plus prompt-injection blocking; flagged text can be redacted.
Computational cost	Token metering, response caching, task-based model routing, targeted (field-level) repair, a session budget cap and a $0 offline/local engine.
Human creative control	Nothing is exported without explicit designer approval; designers edit any field, choose between 3 concept-art variations or upload their own artwork; every AI and human version is stored with author, score and a diff.

4.1 Consistency rules

ID	Rule	Severity
R1	Abilities use only the faction's elements	error
R2	No technology beyond the world's tech level	error
R3	No contradictions with established lore	error
R4	Colour palette matches the faction	warning
R5	Name follows faction naming conventions	warning
R6	Dialogue lines respect the length limit	warning
R7	Character is placed in a valid faction, class and role	error

Score = 100 - 20 x errors - 7 x warnings. Errors block approval unless the designer explicitly overrides and takes responsibility, which is recorded in the version history.

4.2 Why an offline engine?

Real LLMs make characteristic mistakes: they mix up magic systems, add anachronistic technology, contradict lore, reuse famous names and copy artists' styles. The offline engine reproduces these mistakes on purpose. The higher the creativity (temperature) setting, the more often it does so. This makes the guardrails demonstrable and testable without an API key. The same pipeline runs unchanged on a local open-source LLM through Ollama.

5. Evaluation

Each temperature setting was run on 60 random briefs (random faction, class, role and personality). 'Draft'

is the raw generator output; 'final' is after the automatic repair loop.

Temp.	Drafts with violations	Avg draft score	Avg final score	Finals with errors	Avg repair rounds
0.3	0%	100.0	100.0	0%	0.0
0.6	63%	92.1	100.0	0%	0.63
0.8	98%	74.0	100.0	0%	0.98
1.0	100%	54.7	100.0	0%	1.0

At the highest creativity, 100% of raw drafts broke at least one rule (average score 54.7), yet after auto-repair the average score was 100.0 with 0% of characters still containing errors. Higher creativity costs more because more repair calls are needed. That is a real trade-off for the studio to tune.

Guardrail accuracy (test sets)

Test	Result
Unsafe / injection briefs blocked	10/10
Safe briefs wrongly blocked	0/10
Copycat names flagged (e.g. Pikachoo, Kratoz)	10/10
Original generated names wrongly flagged	1/132

The one borderline flag ('Maro' is 89% similar to 'Mario') shows why copyright results are flags for a human to judge rather than automatic blocks.

Cost

With task-based routing (small model for repair and dialogue, medium model for creative work) the estimated cost was $0.0075 per character versus $0.0732 if the largest model were used for everything, about 10x cheaper. Repeated requests are served from cache at $0, and the offline/local engine costs $0. Prices are illustrative tiers configurable in the app.

Automated tests

8 automated tests (pytest) cover every faction/class/personality combination, drift-and-repair, input blocking, copyright detection, caching, the budget cap, dialogue/story checks, the review workflow, export, and a headless run of the full Streamlit UI.

6. Ethics and responsible use

- Human authorship: AI output is a draft; designers approve, edit or reject, and every change is attributed.
- IP respect: the system discourages imitation of named artists or studios and flags look-alike characters.
- Age-appropriate content enforced at input and output; prompt-injection attempts are blocked.
- Transparency: provenance (AI vs human, version, score) is exported with every character.
- Sustainability: caching, routing and targeted repair reduce compute use.

7. Limitations and future work

- Keyword and pattern filters can miss paraphrases. A trained moderation model and embedding-based similarity search would improve recall.
- Copyright checks use a sample list of protected names; a production system would use a licensed database.

- Integrate an image model (e.g. a local Stable Diffusion) to turn the concept-art prompt into images, with the same style guardrails.
- Multi-user review with roles (writer, art director, legal) and comment threads.
- Fine-tune a small model on the studio's own approved characters for a consistent house voice.

8. Conclusion

The project shows that GenAI can be used productively in character design when it is surrounded by the right engineering: grounding in a Game Bible, automated verification and repair, safety and copyright guardrails, cost controls, and a human who always has the final say. The result is faster ideation without giving up quality, safety or creative ownership.

9. How to run

1) Install Python 3.10+. 2) Double-click run.bat (or: pip install -r requirements.txt, then streamlit run app.py). 3) Optional: install Ollama and run 'ollama pull llama3.2' to use a local LLM. 4) Tests: python -m pytest -q

References

- PEGI - Pan European Game Information age rating system, pegi.info.
- Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST '23.
- Streamlit documentation, docs.streamlit.io.
- Ollama - run open-source large language models locally, ollama.com.